

Proyecto 1: UAV Control - Challenge

12.1 Descripción

Este proyecto consiste en el desarrollo de un sistema avanzado de control y monitoreo remoto con un UAV, fundamentado en una arquitectura modular utilizando ROS2. El sistema establece una comunicación bidireccional que permite la ejecución de misiones de vuelo autónomas y la visualización en tiempo real de telemetría crítica, como el nivel de batería, velocidad y altitud. Para garantizar la seguridad del equipo, integra un sistema de failsafe que fuerza un aterrizaje automático si la energía alcanza niveles críticos o el UAV presenta un comportamiento no deseado. Además, el proyecto destaca por incorporar capacidades de visión computacional mediante OpenCV, procesando el flujo de video en vivo para detectar, aislar y publicar el conteo de objetos de color rojo o negro utilizando máscaras en el espacio de color HSV. Toda esta infraestructura de software se encuentra empaquetada dentro de un contenedor Docker, garantizando un entorno de ejecución completamente reproducible, escalable y aislado.

12.2 Objetivos

- Establecer una comunicación bidireccional entre el entorno ROS2 y el dron Tello de DJI.
- Monitorear y mostrar datos críticos de telemetría del dron, como la velocidad, el nivel de batería y la altitud.
- Implementar la visualización en tiempo real del streaming de video capturado por la cámara del dron.
- Crear un nodo de seguridad autónomo que gestione el aterrizaje automático del dron cuando el nivel de batería sea críticamente bajo (e.j., $< 50\%$).
- Programar y ejecutar una misión autónoma simple que incluya despegue, movimiento en el eje X y aterrizaje.
- Contenerizar toda la aplicación ROS 2 utilizando Docker para asegurar un entorno de ejecución reproducible.
- Documentar detalladamente el código y el proceso de desarrollo, explicando cada paso tomado para la implementación.
- Desarrollar un sexto nodo que utilice OpenCV para detectar y contar objetos de color rojo o negro en el video.

12.3 Marco conceptual

Para el desarrollo de este proyecto, es fundamental comprender los conceptos.

1. **ROS2 (Robot Operating System 2):** Es un conjunto de bibliotecas de software y herramientas para construir aplicaciones robóticas. Su arquitectura se basa en nodos,

tópicos, mensajes, servicios y acciones.

- **Nodos (Nodes):** Unidades de procesamiento que realizan una tarea específica. En este proyecto, cada funcionalidad (video, telemetría, etc.) será un nodo.
 - **Tópicos (Topics):** Canales de comunicación (buses) a través de los cuales los nodos intercambian mensajes. Un nodo "publica" mensajes en un tópico y otro "se suscribe" para recibirlos.
 - **Mensajes (Messages):** Estructuras de datos con un tipo definido que se utilizan para la comunicación en los tópicos.
 - **Servicios (Services):** Un mecanismo de comunicación síncrona para interacciones de tipo solicitud/respuesta.
 - **Acciones (Actions):** Un mecanismo para tareas de larga duración que requieren retroalimentación, como la ejecución de una misión.
2. **DOCKER:** Es una plataforma de software que permite crear, probar y desplegar aplicaciones rápidamente a través de contenedores.
- **Contenedor (Container):** Un entorno de ejecución ligero, portátil y aislado que incluye una aplicación y todas sus dependencias.
 - **Dockerfile:** Un script de texto con instrucciones para construir una imagen de Docker.
 - **Redes en Docker:** Es crucial configurar la red del contenedor (por ejemplo, usando `network_mode: host` para permitir la comunicación entre los nodos ROS2 dentro del contenedor y el dron, que se encuentra en la red del anfitrión).
3. **OpenCV (Open Source Computer Vision Library):** Es una biblioteca de programación de código abierto que contiene algoritmos para visión por computadora en tiempo real. Se utilizará para procesar el video del dron, identificar colores y contar objetos.

12.3.1 Introducción

El presente proyecto tiene como finalidad el desarrollo de un sistema modular y robusto para el control, monitoreo y operación autónoma de un dron, el cual puede integrarse y funcionar como parte de una red de sensores, actuando como un nodo sensor de imágenes. Utilizando ROS2, se busca crear una arquitectura de software desacoplada en la que cada funcionalidad principal se encapsule en un nodo independiente.

Para garantizar la portabilidad, la escalabilidad y un entorno de desarrollo consistente, toda la aplicación se ejecutará dentro de un contenedor Docker. Este enfoque no solo simplifica la configuración del entorno de trabajo, sino que también facilita el despliegue del sistema en diferentes máquinas.

El proyecto abarca desde el diseño de la comunicación y la visualización de datos hasta la implementación de lógica autónoma para la ejecución de misiones. Además, se propone un módulo de inteligencia artificial para el reconocimiento de objetos, estableciendo las bases para futuras aplicaciones avanzadas en redes de sensores basadas en visión por computadora.

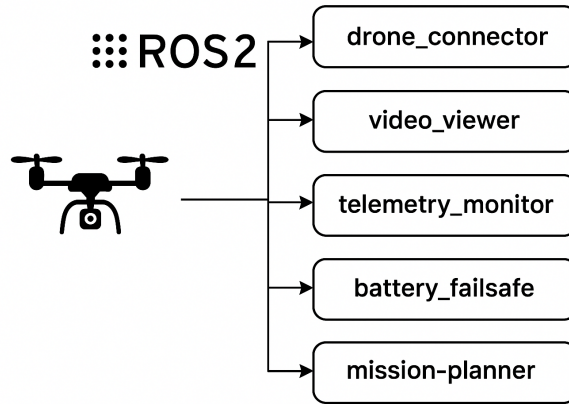


Figure 12.1: Nodos ROS del proyecto

12.4 Alcance del Proyecto

El proyecto se dividirá en el desarrollo de los siguientes nodos y componentes (ver Fig. 13.1):

- **Nodo 1: Comunicación con el Dron (drone_connector):** Este nodo será el puente de comunicación entre el sistema ROS2 y el dron. Su única responsabilidad es establecer y mantener una conexión estable. Se utilizarán paquetes de ROS2 para el dron Tello. El nodo deberá verificar continuamente el estado de la conexión y reportarlo.
- **Nodo 2: Visualización de Video (video_viewer)** El nodo debe Mostrar el video de la cámara del dron en una ventana en tiempo real. El nodo se suscribirá al tópico de ROS2 que transmite las imágenes de la cámara tipo `sensor_msgs/msg/Image`. Se utilizará la librería `cv_bridge` para convertir los mensajes de imagen de ROS2 a un formato compatible con OpenCV. Se usará la función `cv2.imshow()` de OpenCV para mostrar el video en una ventana. El nodo muestra una ventana en la que se observa el feed de video en vivo del dron.
- **Nodo 3: Monitor de Telemetría (telemetry_monitor):** El nodo debe obtener y mostrar los datos de telemetría más importantes del dron y crear suscriptores para los tópicos relevantes, como nivel de batería, velocidad, altura, etc. El nodo formateará estos datos y los imprimirá en la consola de manera clara y legible, actualizándose constantemente.
- **Nodo 4: Lógica de Batería Segura (battery_failsafe):** El nodo actúa como un sistema de seguridad autónomo basado en el nivel de batería. El nodo se suscribirá al tópico de estado de la batería e implementará una lógica condicional; si el porcentaje de batería es inferior al 30%, el nodo publicará un comando de aterrizaje en el tópico de control del dron. Además, antes de que se pueda enviar un comando de despegue (desde el nodo de misión), este nodo deberá validar que la batería esté por encima del umbral.
- **Nodo 5: Misión Planner (mission-planner):** El nodo debe ejecutar una secuencia

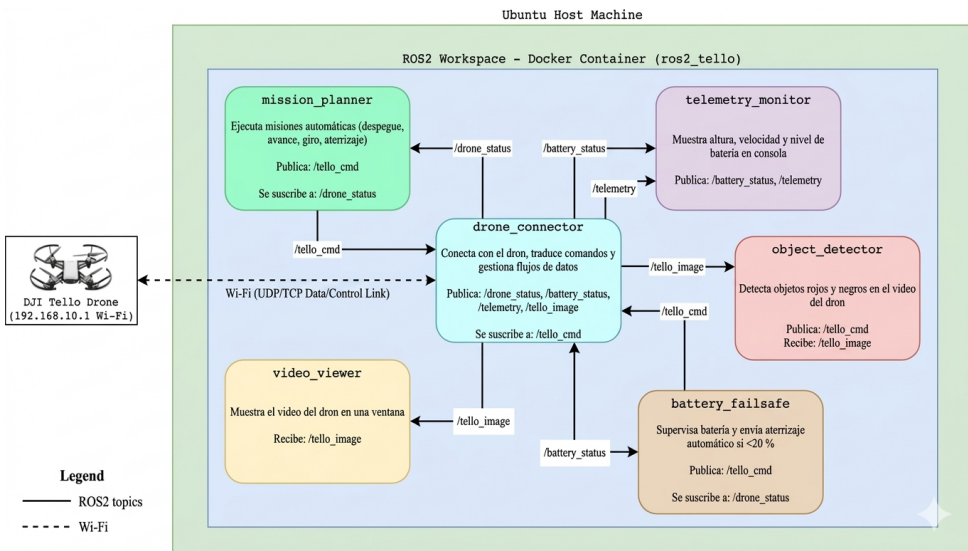


Figure 12.2: Ejemplo de la arquitectura de la aplicación de control de dron.

de vuelo autónoma predefinida. Se publicará una serie de comandos de control. La secuencia de la misión será:

1. Enviar el comando de despegue.
2. Esperar a alcanzar una altura de 50 cm.
3. Crear demoras de 3 a 5 segundos para la estabilización del sistema y del equipo dron.
4. Enviar el comando para avanzar 50 cm hacia adelante en el eje X.
5. Pausar durante un tiempo configurable.
6. Enviar comando para retroceder 50 cm (volver a la posición original).
7. Pausar durante un tiempo configurable.
8. Enviar comando de aterrizaje.

12.4.1 Requisito Clave: Integración con Docker

Todos los nodos deben correr dentro de un único contenedor de Docker. Se debe investigar cómo configurar la red de Docker para permitir la comunicación entre el contenedor y el dron. Generalmente, esto se logra utilizando el modo de red 'host' (en windows suele requerir pasos adicionales), que hace que el contenedor comparta la pila de red del anfitrión, como hemos realizado a lo largo de los Talleres. La Fig. 13.2 muestra un ejemplo de la interacción de los nodos en ROS y el hardware externo.

12.4.2 Recomendaciones:

- Mantener el orden del espacio de trabajo (workspace) de ROS2 del proyecto, creado por el comando `ros2 pkg create`, tanto en carpetas como en archivos.
- Se recomienda usar `docker-compose` para automatizar el proceso de despliegue del

contenedor.

- Cada nodo debe ejecutarse en una pantalla distinta para que se pueda observar el video, los valores de telemetría, el estado de conexión al dron y el estado de la batería.
- Instalar todas las dependencias necesarias: `python3`, `opencv`, `cv-bridge`, etc.
- Tomar en cuenta la compatibilidad de drivers para ejecutar ventanas gráficas para el nodo de video, se sugiere seguir los siguientes pasos:

1. Ejecute en la terminal de su sistema (no dentro del contenedor) y copie el valor que le salga (por ejemplo :0 o 1); vamos a usarlo dentro del contenedor.

```
echo $DISPLAY.
```

2. Permita el acceso desde Docker al servidor gráfico X11 ejecutando

```
xhost +local:docker.
```

3. Entra al contenedor. si ya está corriendo, use `'docker exec -it <nombre del contenedor> bash '`, o si va a ejecutarlo nuevamente, `docker start -ai <nombre> ;` dentro del contenedor, configure las variables de entorno (si el comando `display` le mostró otro valor, reemplace por **0**):

```
export DISPLAY=:0
export QT_X11_NO_MITSHM=1
```

4. Como prueba de la conexión gráfica, dentro del contenedor instalaremos un miniprograma de prueba llamado `xeyes`. si devuelve una ventana grafica con ojos, estaremos con conexión correcta a la interfaz gráfica.

```
apt update && apt install -y x11-apps
xeyes
```

5. Por ultimo, debemos entender que docker no reconoce por defecto nuestra tarjeta gráfica ni los aceleradores graficos de la gpu de nuestra maquina host, por lo que hay que deshabilitarlos dentro de docker para que no causen conflictos. Para ello, en el codigo que maneje el nodo de video, incluiremos las siguientes funciones al principio del código, antes de la lectura de las imagenes :

```
"""Incluir estas lineas al inicio de todo el nodo que lee el
    video para que OpenCV use la CPU sin render acelerado y sin
    OpenGL"""

import os
os.environ["QT_QPA_PLATFORM"] = "xcb"
os.environ["LIBGL_ALWAYS_SOFTWARE"] = "1" # render por CPU
os.environ["XDG_RUNTIME_DIR"] = "/tmp"
```

6. Otro método para evitarnos los pasos anteriores al ejecutar el nodo de video es, al momento de desplegar por primera vez el contenedor, incluir las opciones `DISPLAY`, `XDG_RUNTIME_DIR`, `X11-unix`, `/dev/dri`, que cumplen las mismas funciones que las lineas agregadas al código. Sin embargo, esta solución depende de las capacidades de la maquina para su funcionamiento. Se detalla un ejemplo de esta configuración :

```
docker run -it \  
  --name ros_tello \  
  --network host \  
  -e DISPLAY=$DISPLAY \  
  -e XDG_RUNTIME_DIR=/tmp \  
  -v /tmp/.X11-unix:/tmp/.X11-unix \  
  --device /dev/dri:/dev/dri \  
  ros_tello_image:latest
```

12.4.3 Nodo de reconocimiento de Objetos (Nodo 6):

Analizar el video de la cámara para detectar y contar objetos de color rojo o negro. Se debe suscribir al tópico de la imagen de la cámara. Convertir la imagen del espacio de color BGR a HSV, que es más robusto para la detección de colores. Definir los rangos de color para el rojo y el negro en el espacio HSV. Aplicar una máscara a la imagen para aislar solo los píxeles que caen dentro de esos rangos de color. Utilizar la función `cv2.findContours()` para encontrar los contornos de los objetos aislados. Filtrar los contornos por área para eliminar el ruido. Dibujar un rectángulo `cv2.rectangle()` alrededor de cada objeto detectado y mostrar un contador en la pantalla. Publicar el número de objetos detectados en un nuevo tópico de ROS2, tipo `std_msgs/msg/Int32`, para mostrarlos en la pantalla.

12.4.4 Métricas de Rendimiento (Performance Metrics)

Para asegurar que el sistema es robusto y eficiente, se deben medir los siguientes aspectos durante la ejecución de los nodos:

Métricas del Sistema y Red (Docker & ROS2)

- **Latencia de Tópicos:** Medir el tiempo de retraso entre el momento en que el `drone_connector` publica los datos de telemetría y el momento en que el `telemetry_monitor` los procesa y muestra.
- **Pérdida de Paquetes (Packet Loss):** Evaluar la estabilidad de la conexión Wi-Fi entre el dron Tello y la máquina anfitriona, midiendo la pérdida de paquetes de control y video.
- **Consumo de Recursos (Docker):** Medir el uso de CPU y memoria RAM del contenedor Docker durante la ejecución completa es crucial, dado que se fuerza el renderizado de video.

Métricas de Visión Computacional (OpenCV)

- **Tasa de Fotogramas (FPS):** Medir los FPS reales que el nodo `video_viewer` logra mostrar en pantalla, comparado con los FPS que envía el dron.
- **Precisión de Detección:** Contabilizar la tasa de falsos positivos y falsos negativos al detectar y contar objetos de color rojo o negro bajo diferentes condiciones de iluminación.
- **Latencia de Procesamiento:** Medir el tiempo que le toma al nodo `object_detector` convertir la imagen de BGR a HSV, aplicar las máscaras, encontrar contornos y publicar el conteo.

12.5 Rúbrica de Evaluación

Esta rúbrica permite evaluar los entregables finales (informe, código fuente y presentación) de forma objetiva.

Categoría	Tarea a Evaluar	Puntos Máximos
Misión Autónoma	Completar el circuito predefinido (despegue, navegación, aterrizaje) sin asistencia manual. Se premia el menor tiempo de reacción en ir y volver	30 pts
Precisión de Vuelo	Se acerca al punto de inicio. Penalización por comportamiento errático.	20 pts
Visión Computacional	Conteo exacto de objetos rojos y negros. Penalización por cada falso positivo o falso negativo.	30 pts
Failsafe (Stress Test)	Reacción ante la inyección de un comando de batería crítica simulado. Se premia el menor tiempo de reacción en forzar el aterrizaje.	10 pts
Eficiencia y Recursos	Menor consumo de recursos (CPU/RAM) del host medido con <code>docker stats</code> durante el vuelo.	10 pts
PUNTUACIÓN TOTAL:		100 pts

12.5.1 Entregables Finales del Proyecto:

1. **Informe del proyecto 1:** En el informe, explique paso a paso la arquitectura del sistema. Debe adjuntar el código fuente con un workspace completo de ROS2 que contenga todos los paquetes y nodos desarrollados. Además, se debe subir la imagen de docker a la nube Docker. El código de cada nodo debe contener comentarios claros y las instrucciones sobre cómo compilar y ejecutar el proyecto.
2. **Presentación del proyecto 1:** Finalmente, se generará una presentación de demostración del sistema funcionando, que incluirá la visualización del video, la telemetría, la ejecución de la misión, el fail-safe de la batería y el reconocimiento de objetos.